Policy e Value Iterations

a.a. 2023/2024

Mario Sassano

Dipartimento di Ingegneria Civile e Ingegneria Informatica
Università di Roma Tor Vergata

Nelle lezioni precedenti...

- Abbiamo definito i concetti di Funzione valore di stato e Funzione valore di stato/azione
- Abbiamo visto che la funzione valore di stato permette un ordinamento parziale delle policies
- Abbiamo visto che la conoscenza della Funzione valore di stato/azione *risolve* il problema decisionale



risolvendo il problema ad un passo sui controlli possibili in ogni stato x. SI ELABORA UNA POLITICA DECISIONALE IN RETROAZIONE DALLO STATO → come nella PD, indipendentemente dalla coda (come il gruppo conosco l'azione ad un passo ottima nel lungo percorso.

Policy Evaluation

Data una policy π, vogliamo calcolare il suo valore v_π da ciascuno stato

$$v_\pi(x) = R_{k+1} + \gamma R_{k+2} + \gamma^2 R_{k+3} + \ldots = R_{k+1} + \gamma v_\pi(x') \tag{1}$$

dove $x' = f(x, \pi(x))$ è il successore di x seguendo π

Abbiamo visto che si tratta di un sistema di $|X|$ equazioni lineari.

Tuttavia, inversione di matrice può essere *computazionalmente onerosa*

Approssimazione iterativa (*)

Consideriamo la *sequenza* $v_0, v_1, v_2, \ldots$ di approssimazioni di v_π, $v_i : X \to \mathbb{R}^m$, con v_0 *arbitraria* tranne $v_0(x) = 0$ per ogni $x \in X^+$ (stati terminali), definita da

$$v_{i+1}(x) = \underbrace{R_{k+1}}_{\varrho(x,\pi(x))} + \gamma v_i(x'), \qquad \forall x \in X, \quad i = 0, 1, 2, \ldots$$

- se ad un passo non aggiorniamo più il valore di nessuno stato, ovvero $v_{i+1}(x) = v_i(x)$ per ogni x, allora v_i è un *punto fisso* e risolve (1)
- (si dimostra che $v_i \to v_\pi$ per $i \to \infty$)

(*) Anche per calcolare v_π (non ottima) di una certa π uso una procedura iterativa che genera una sequenza.

Commenti su Policy Evaluation

Approssimazione iterativa

Consideriamo la *sequenza* v_0, v_1, v_2, ... di approssimazioni di v_π, $v_i : X \to \mathbb{R}$, con v_0 arbitraria tranne $v_0(x) = 0$ per ogni $x \in X^+$ (stati terminali) definita da

$$(\Sigma_u) \qquad \mathbf{v}_{i+1} = \mathcal{R} + \gamma \mathcal{T}_\pi \mathbf{v}_i, \qquad i = 0, 1, 2, \ldots$$

Da un punto di vista di codice:

- apparentemente abbiamo bisogno di due vettori per salvare v_i e v_{i+1} (*full backup*), ma possiamo anche usarne solo uno utilizzando i valori $v_i(x')$ se già disponibili (ovvero se x' è stato aggiornato prima di x...)
- abbiamo bisogno di uno *stopping criterion*: $\max_{x \in X} |v_{i+1}(x) - v_i(x)| < \varepsilon$, con $\varepsilon > 0$ sufficientemente piccolo (aggiorniamo qualche stato, ma di poco...)

→ il sistema è libero e converge a 0 ($|\lambda| < 1 \ \forall \lambda \in \sigma(\gamma T_\pi)$).

Σ_u è un sistema a tempo discreto asintoticamente stabile; la sua matrice dinamica è γT_π (matrice con al più un el. su ogni riga < 1)

BISOGNEREBBE DIMOSTRARE CHE γT_π È STABILE

Aggiornamento con un solo vettore.

↓

aggiorno $V_{i+1}(x^j)$ $j = 1 \dots m$ ogni volta.

V stato utilizzato non solo l' inf. a i ma anche quella di $i+1$ se il successore x` è stato aggiornato.

Algoritmo per Policy Evaluation

Approssimazione iterativa

Consideriamo la *sequenza* v_0, v_1, v_2, ... di approssimazioni di v_π, $v_i : X \to \mathbb{R}$, con v_0 arbitraria tranne $v_0(x) = 0$ per ogni $x \in X^+$ (stati terminali) definita da

$$v_{i+1}(x) = \underbrace{R_{k+1}}_{\varrho(x,\pi(x))} + \gamma v_i(x'), \qquad \forall x \in X, \quad i = 0, 1, 2, \ldots$$

Policy Evaluation (PE) → problema che serve per stimare v_π

→ ALG. DI APPROX ITERATIVA

→ INVERSIONE DELLA MATRICE.

Input: π, policy da valutare, $\varepsilon > 0$

Inizializzazione: $V(x)$ tale che $V(x) = 0$, $\forall x \in X^+$, $\Delta = \varepsilon$

Iterazioni

```
while Δ ≥ ε                                   ← Δ = 0.
    for each x ∈ X
        v = V(x)
        V(x) = ϱ(x, π(x)) + γ V(x')           ← dove x' è il successore di x
        Δ = max{Δ, |v − V(x)|}                  secondo la T_π.
    end
end
```

Policy Improvement (1/3) → per ora lo abbiamo solo calcolato, ora lo voglio migliorare

Dopo aver calcolato il valore della policy π, vorremmo cercare di **migliorarlo**

Supponiamo di avere una policy π e di aver calcolato il suo valore v_π

$\Rightarrow$ ci chiediamo se per qualche stato non sarebbe meglio una azione $u \neq \pi(x)$

Policy Improvement (PI)

Possiamo implementare u nello stato x e poi *seguire* π, ovvero sfruttare $q_\pi(x, u)$:

$$q_\pi(x, u) = \underbrace{R_{k+1}}_{\text{con } u} + \gamma \overbrace{v_\pi}^{\text{con } \pi}\left(\underbrace{x'}_{\text{con } u}\right)$$

$q_\pi(x, u) = \max$

Se $q_\pi(x, u) \geqslant v_\pi(x) \Rightarrow$ conviene eseguire u in x invece di $\pi(x)$!

conviene cambiare la politica puntualmente. Ma se esistesse un' altra u : $q_\pi(x, u) > q_\pi(x, u)$?

Policy Improvement (PI)

Possiamo implementare u nello stato x e poi *seguire* π, ovvero sfruttare $q_\pi(x, u)$:

$$q_\pi(x, u) = \underbrace{R_{k+1}}_{\text{con } u} + \gamma \overbrace{v_\pi}^{\text{con } \pi}\left(\underbrace{x'}_{\text{con } u}\right)$$

Se $q_\pi(x, u) \geqslant v_\pi(x) \Rightarrow$ conviene eseguire u in x invece di $\pi(x)$!

$\Rightarrow$ definiamo $\hat{\pi}$ uguale a π tranne in x, $\hat{\pi}(x) = u$

Eseguiamo la stessa operazione per ogni stato ridefinendo l'intera policy $\hat{\pi}$!

$$\begin{aligned} \hat{\pi}(x) &= \arg\max_u q_\pi(x, u), \qquad \forall x \in X \\ &= \arg\max_u \{\varrho(x, u) + \gamma v_\pi(f(x, u))\} \end{aligned} \tag{2}$$

$\hat{\pi}$ è definita aggiornando π in modo **greedy** rispetto a q_π

Policy Improvement (3/3)

Abbiamo quindi che:

$$\hat{\pi} \geqslant \pi$$

la $\hat{\pi}$ è calcolata ad un passo (greedy) ed è un ottimo locale data la π arbitraria.

Infatti: siano π e $\hat{\pi}$ due policies, se vale che

$$q_\pi(x, \hat{\pi}(x)) \geqslant v_\pi(x), \qquad \forall x \in X$$

allora

$$v_\pi(x) \leqslant q_\pi(x, \hat{\pi}(x)) = \overbrace{R_{k+1}}^{\text{con } \hat{\pi}(x)} + \gamma v_\pi(x')$$

ottenuto da x tramite $\hat{\pi}(x)$ → infinite ricompense ottenute con π da x'.

$$\leqslant R_{k+1} + \gamma q_\pi(x', \hat{\pi}(x')) = R_{k+1} + \gamma \overbrace{R_{k+2}}^{\text{con } \hat{\pi}(x')} + \gamma^2 v_\pi(x'') \leqslant \ldots \leqslant v_{\hat{\pi}}(x) \qquad \forall x \in X.$$

per def.

ottenuta sfruttando la disuguaglianza iterativamente per ogni stato

Algoritmo di Policy Iteration

Criterio di arresto: se $\nexists x \in X$ per cui $v_{\hat{\pi}}(x) > v_{\pi}(x)$, allora $v_{\hat{\pi}} = v_{\pi}$ è **punto fisso** di (2) $\Rightarrow v_{\hat{\pi}}(x) = v_{\pi}(x) = v_{\star}(x)$, $\forall x \in X$, Funzione Valore ottima

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} \cdots \longrightarrow v_{\star}$$

Policy Iteration (PI)

Inizializzazione: $V(x)$, $\pi(x)$

Iterazioni

1. **Policy Evaluation** → con approx. iterative.
2. **Policy Improvement**

policy-stable = TRUE (non ho aggiornato nessuna decisione in nessuno stato)

for each $x \in X$

$a = \pi(x)$

$\pi(x) \in \arg\max_u \{\varrho(x,u) + \gamma V(f(x,u))\}$

if $a \neq \pi(x)$, then *policy-stable* = FALSE

if *policy-stable*, stop and return V, π

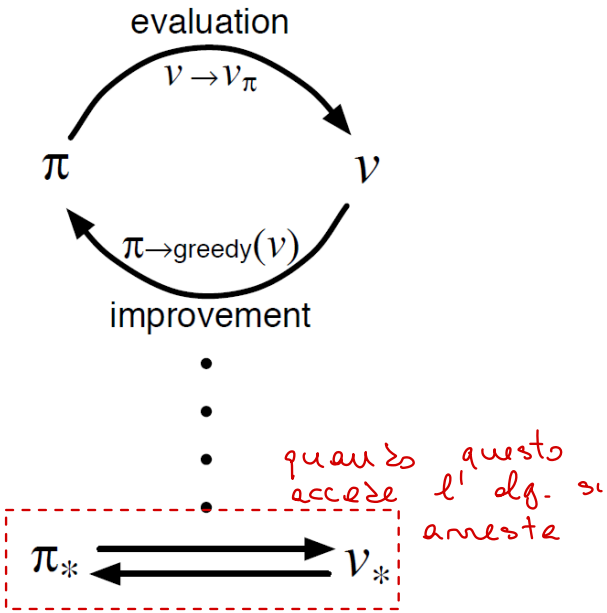
else goto 1. Policy Evaluation

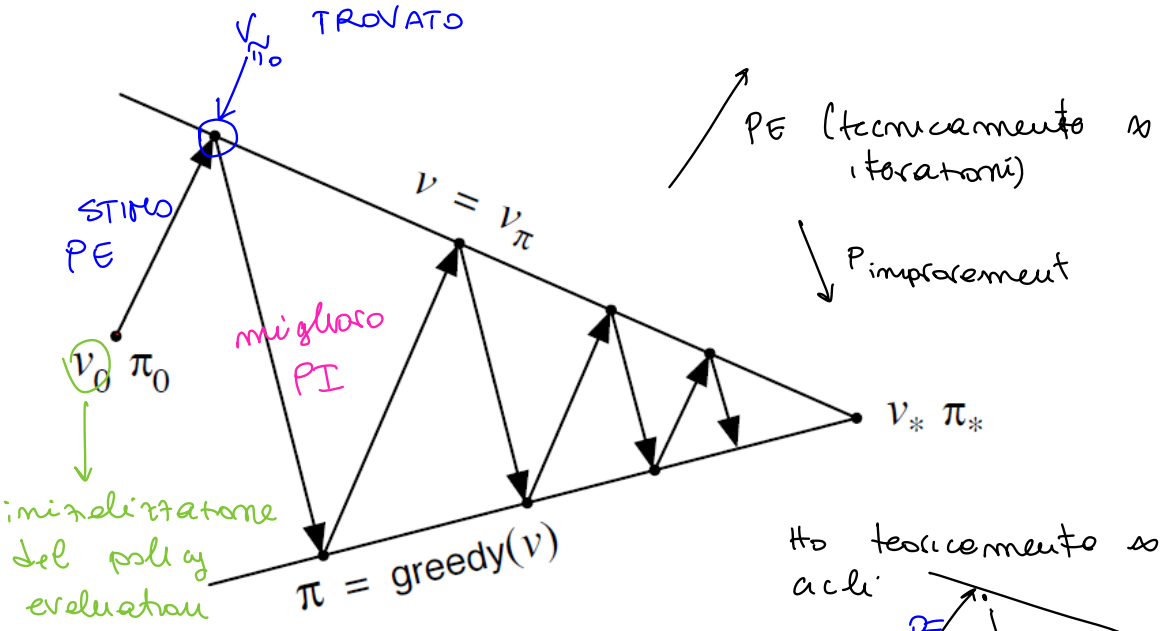
Commenti su Policy Iteration

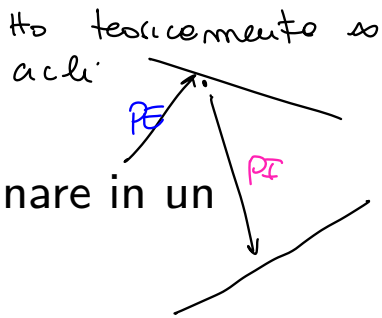
Inconveniente di Policy Iteration

In linea teorica si ha ∞ cicli di ∞ it. per la PE. (α^2) $\Rightarrow$ devo ridurre.

Ad ogni passo di PI è necessario eseguire anche una PE!







Inoltre, se eseguiamo PE *iterativamente* dovremo **realisticamente** terminare in un numero **finito** di iterazioni

$\Rightarrow$ **PI converge anche eseguendo PE iterativo "troncato"**

Policy Iteration troncato

Eseguiamo PI svolgendo un numero finito di aggiornamenti per PE

i-esimo passo di PI troncato

Data $\pi_i(x)$

PE troncato: data v_0^{PE} arbitraria

$$v_{j+1}^{PE}(x) = \varrho(x, \pi_i(x)) + \gamma v_j^{PE}(x'), \ \forall x \in X, \quad j = 0, \ldots, N-1$$

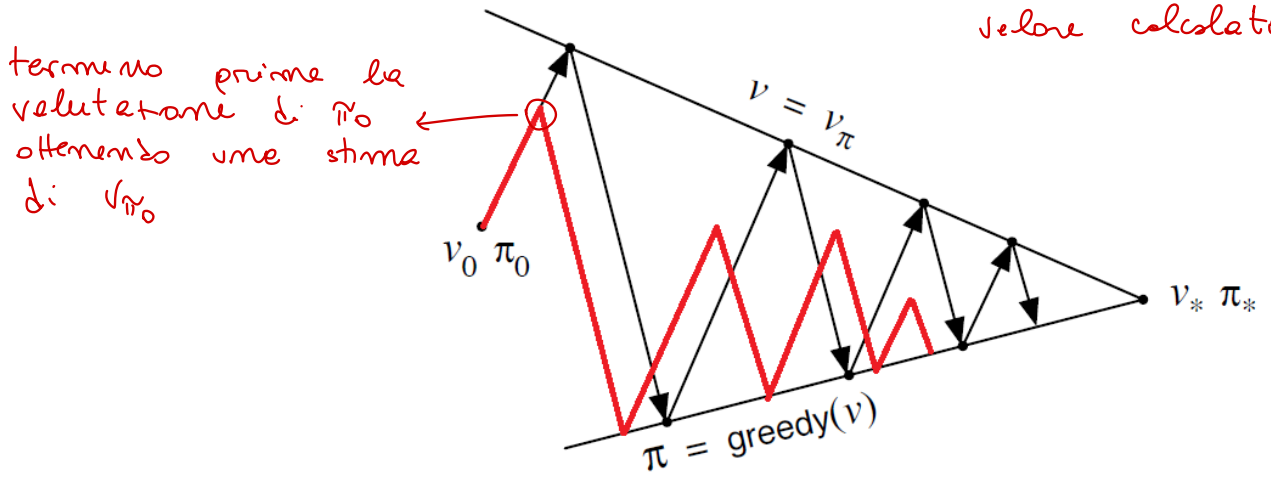
(SE ESEGUO N PASSI NON SO)

Policy Improvement:

$$\pi_{i+1}(x) = \arg\max_u \{\varrho(x,u) + \gamma v_N^{PE}(f(x,u))\} \Rightarrow$$

sono nel discreto e il PI converge prime

valore calcolato stimato di v_{π_i}.



Che succede se fissiamo $N = 1$ (facciamo una sola iterazione di PE)?

Data $v_i(x)$
Policy Improvement:

$$\pi_{i+1}(x) = \arg\max_u \{\varrho(x,u) + \gamma v_i(f(x,u))\} \quad (1)$$

PE troncato ad una iterazione:

$$v_{i+1}(x) = \varrho(x, \pi_{i+1}(x)) + \gamma v_i(f(x, \pi_{i+1}(x))), \ \forall x \in X \quad (2)$$

Value Iteration

L'equazione di Bellman si trasforma in una **legge di aggiornamento** (non è un'eq. ricorsiva perché a dx ho termini noti)

$$v_{i+1}(x) = \max_u \{R_{k+1} + \gamma v_i(x')\}, \quad \forall x \in X$$

(non è una eq. con la var. nel max)

(non ho più π e parlo solo di valore come in Bellman.)

tale che $v_i \to v_\star$, per $i \to \infty$

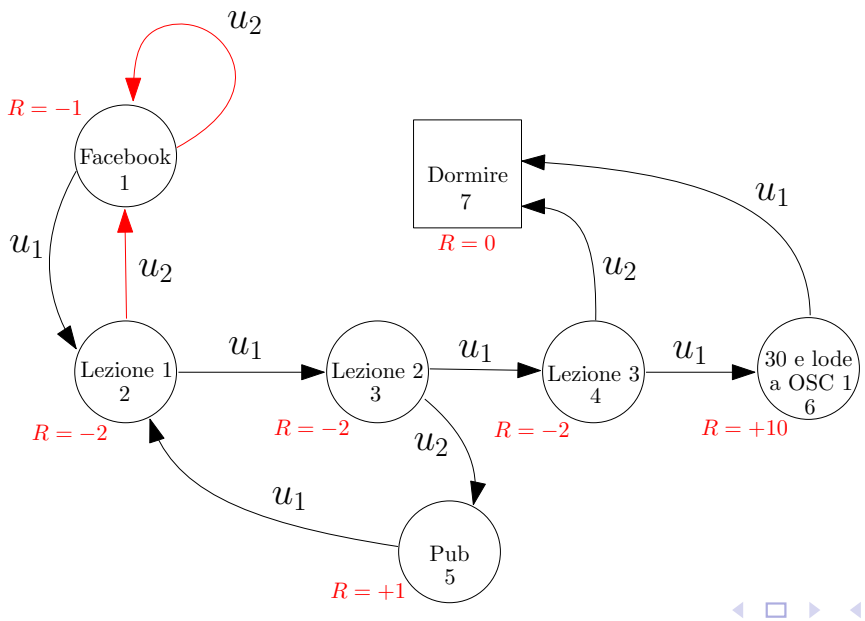
value iteration funziona solo se conosco il modello. { ricostruisco π a posteriori con p e f. Se p ed f non sono noti mi' attacco

Esempio: comportamento ottimo dello studente

Consideriamo la policy dello studente che si diverte e cerchiamo di migliorarla...

$$\pi_0 = \begin{bmatrix} u_2 \\ u_2 \\ u_2 \\ u_2 \\ u_1 \\ u_1 \\ \star \end{bmatrix} \Rightarrow v_{\pi_0} = \begin{bmatrix} -2 \\ -3 \\ -2.25 \\ -2 \\ -0.5 \\ 10 \\ 0 \end{bmatrix} \Rightarrow q_{\pi_0} = \begin{bmatrix} -2.5 & -2 \\ -3.125 & -3 \\ -3 & -2.25 \\ 3 & -2 \\ -0.5 & \star \\ 10 & \star \\ 0 & \star \end{bmatrix} \Rightarrow \pi_1 = \begin{bmatrix} u_2 \\ u_2 \\ u_2 \\ u_1 \\ u_1 \\ u_1 \\ \star \end{bmatrix}$$

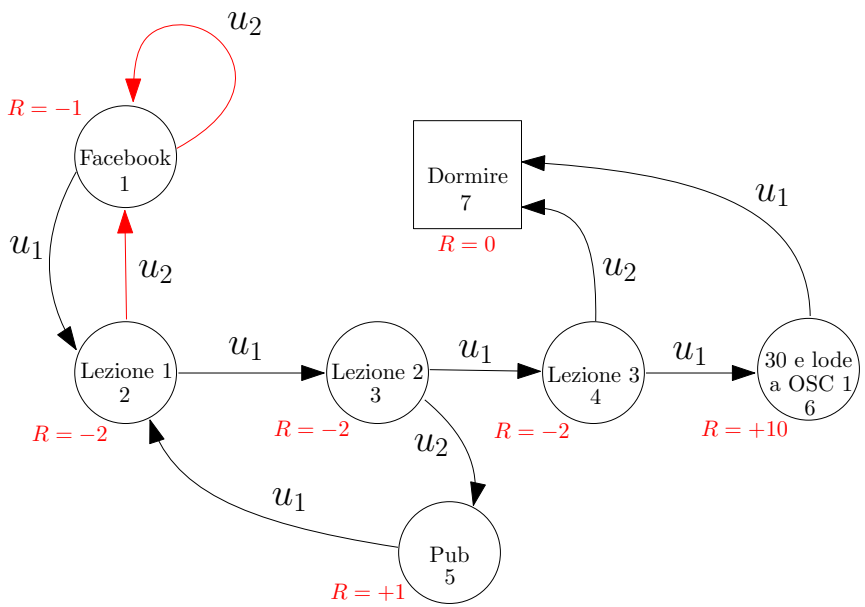
Partendo da "Lezione 1" e seguendo π_1



Esempio: comportamento ottimo dello studente

$$\pi_1 = \begin{bmatrix} u_2 \\ u_2 \\ u_2 \\ u_1 \\ u_1 \\ u_1 \\ \star \end{bmatrix} \Rightarrow v_{\pi_1} = \begin{bmatrix} -2 \\ -3 \\ -2.25 \\ 3 \\ -0.5 \\ 10 \\ 0 \end{bmatrix} \Rightarrow q_{\pi_1} = \begin{bmatrix} -2.5 & -2 \\ -3.125 & -3 \\ -0.5 & -2.25 \\ 3 & -2 \\ -0.5 & \star \\ 10 & \star \\ 0 & \star \end{bmatrix} \Rightarrow \pi_2 = \begin{bmatrix} u_2 \\ u_2 \\ u_1 \\ u_1 \\ u_1 \\ u_1 \\ \star \end{bmatrix}$$

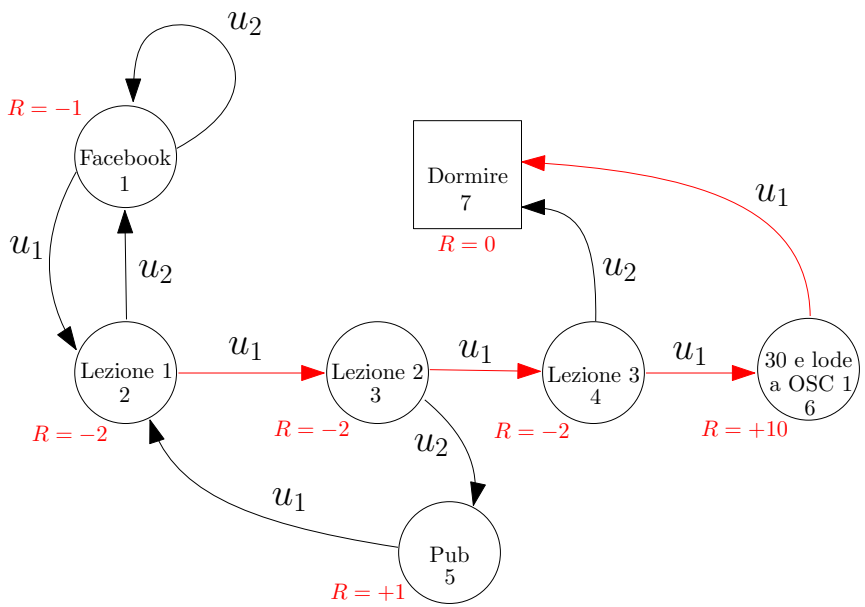
Partendo da "Lezione 1" e seguendo π_2



Esempio: comportamento ottimo dello studente

$$\pi_2 = \begin{bmatrix} u_2 \\ u_2 \\ u_1 \\ u_1 \\ u_1 \\ u_1 \\ \star \end{bmatrix} \Rightarrow \quad v_{\pi_2} = \begin{bmatrix} -2 \\ -3 \\ -0.5 \\ 3 \\ -0.5 \\ 10 \\ 0 \end{bmatrix} \Rightarrow \quad q_{\pi_2} = \begin{bmatrix} -2.5 & -2 \\ -2.25 & -3 \\ -0.5 & -2.25 \\ 3 & -2 \\ -0.5 & \star \\ 10 & \star \\ 0 & \star \end{bmatrix} \Rightarrow \quad \pi_3 = \begin{bmatrix} u_2 \\ u_1 \\ u_1 \\ u_1 \\ u_1 \\ u_1 \\ \star \end{bmatrix}$$

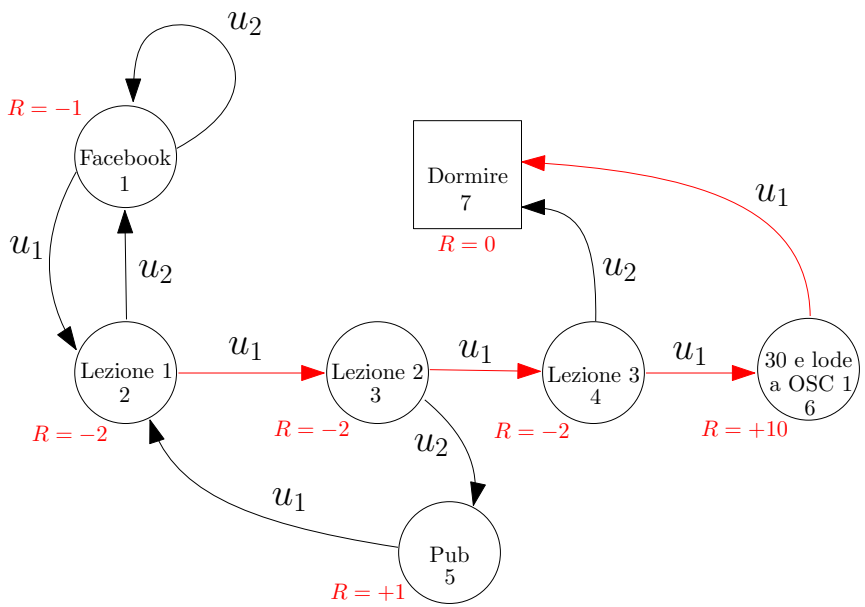
Partendo da "Lezione 1" e seguendo π_3



Esempio: comportamento ottimo dello studente

$$\pi_3 = \begin{bmatrix} u_2 \\ u_1 \\ u_1 \\ u_1 \\ u_1 \\ u_1 \\ \star \end{bmatrix} \Rightarrow \quad v_{\pi_3} = \begin{bmatrix} -2 \\ -2.25 \\ -0.5 \\ 3 \\ -0.125 \\ 10 \\ 0 \end{bmatrix} \Rightarrow \quad q_{\pi_3} = \begin{bmatrix} -2.125 & -2 \\ -2.25 & -3 \\ -0.5 & -2.0625 \\ 3 & -2 \\ -0.125 & \star \\ 10 & \star \\ 0 & \star \end{bmatrix} \Rightarrow \quad \pi_4 = \pi_3 = \pi_\star$$

Partendo da "Lezione 1" e seguendo π_4



Programmazione Dinamica Asincrona

Inconveniente dell'implementazione precedente:

$\Rightarrow$ ad ogni iterazione abbiamo considerato un *full backup* di tutti gli stati ("$\forall x \in X$")

impossibile in alcune situazioni (backgammon, 10^{20} stati con 1 milioni di stati al secondo $\rightarrow$ 1000 anni..)

DP Asincrona: si aggiornano solo gli stati coinvolti nell'episodio appena concluso (mosse degli scacchi in una partita...)

- il valore di alcuni stati (più "frequenti") potrebbe essere aggiornato molte volte prima che altri lo siano anche una sola volta
- bisogna garantire che tutti gli stati siano sufficientemente aggiornati

Nelle prossime lezioni...

siamo interessati a tecniche che permettano di risolvere problemi di RL senza conoscere il modello (sfruttato, ad esempio, nel calcolo della q_{π_i} nell'esempio...)

Prossimi passi:

- studiamo l'estensione al caso stocastico
- studiamo RL basato esclusivamente su esperienza e dati raccolti
- Temporal-difference learning, Q-learning